Combinational Circuits Tutorial

Table of Contents

Introduction

Combinational circuits are digital logic circuits where the output depends only on the current input values. Unlike sequential circuits, they have no memory elements and do not depend on previous states or timing.

Key Characteristics:

- **No memory**: Output depends only on present inputs
- **No feedback loops**: Information flows in one direction
- **Instantaneous response**: Output changes immediately when inputs change
- **No clock required**: Asynchronous operation

Examples of Combinational Circuits:

- Logic gates (AND, OR, NOT, etc.)
- Multiplexers and Demultiplexers
- Encoders and Decoders
- Adders and Subtractors
- Comparators

Basic Logic Gates

Logic gates are the fundamental building blocks of combinational circuits.

1. AND Gate

- **Function**: Output is 1 only when ALL inputs are 1
- **Symbol**: D-shaped with flat input side

- **Boolean Expression**: Y = A · B (or A AND B)

2. **OR Gate**

- **Function**: Output is 1 when ANY input is 1
- **Symbol**: Curved input side, pointed output
- **Boolean Expression**: Y = A + B (or A OR B)

3. **NOT Gate (Inverter)**

- **Function**: Output is opposite of input
- **Symbol**: Triangle with small circle (bubble) at output
- **Boolean Expression**: Y = $\bar{A}$ (or NOT A)

4. **NAND Gate**

- **Function**: AND gate followed by NOT gate
- **Symbol**: AND gate with bubble at output
- **Boolean Expression**: Y = (A · B)' or Y = $\bar{A}$ + $\bar{B}$

5. **NOR Gate**

- **Function**: OR gate followed by NOT gate
- **Symbol**: OR gate with bubble at output
- **Boolean Expression**: Y = (A + B)' or Y = $\bar{A}$ · $\bar{B}$

6. **XOR Gate (Exclusive OR)**

- **Function**: Output is 1 when inputs are different
- **Symbol**: OR gate with additional curved line at input
- **Boolean Expression**: Y = A ⊕ B = A·$\bar{B}$ + $\bar{A}$·B

7. **XNOR Gate (Exclusive NOR)**

- **Function**: Output is 1 when inputs are the same
- **Symbol**: XOR gate with bubble at output
- **Boolean Expression**: Y = (A ⊕ B)' = A·B + $\bar{A}$·$\bar{B}$

Truth Tables

Truth tables show all possible input combinations and their corresponding outputs.

Example: 2-input AND Gate

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Example: 2-input XOR Gate

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Boolean Algebra

Boolean algebra provides mathematical framework for analyzing and simplifying logic circuits.

Basic Laws:

1. **Identity Laws**:
   - $A + 0 = A$
   - $A \cdot 1 = A$
2. **Null Laws**:
   - $A + 1 = 1$
   - $A \cdot 0 = 0$
3. **Idempotent Laws**:
   - $A + A = A$
   - $A \cdot A = A$
4. **Complement Laws**:
   - $A + \bar{A} = 1$
   - $A \cdot \bar{A} = 0$
5. **De Morgan's Laws**:
   - $(A + B)' = \bar{A} \cdot \bar{B}$
   - $(A \cdot B)' = \bar{A} + \bar{B}$

Simplification Example:

Original: $Y = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B$

Step 1: $Y = A \cdot (B + \bar{B}) + \bar{A} \cdot B$

Step 2: $Y = A \cdot 1 + \bar{A} \cdot B$

Step 3: $Y = A + \bar{A} \cdot B$

Common Combinational Circuits

1. Half Adder

Purpose: Adds two single bits

Inputs: A, B

Outputs: Sum (S), Carry (C)

Logic:

- Sum = $A \oplus B$
- Carry = $A \cdot B$

Truth Table:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2. Full Adder

Purpose: Adds two bits plus carry from previous stage

Inputs: A, B, Cin (carry in)

Outputs: Sum (S), Cout (carry out)

Logic:

- Sum = $A \oplus B \oplus Cin$
- Cout = $A \cdot B + Cin \cdot (A \oplus B)$

3. Multiplexer (MUX)

Purpose: Selects one of multiple inputs based on select lines

Example - 4:1 MUX:

- **Inputs**: I0, I1, I2, I3
- **Select lines**: S1, S0

- **Output**: Y

Logic: Y = $\bar{S}1 \cdot \bar{S}0 \cdot I0 + \bar{S}1 \cdot S0 \cdot I1 + S1 \cdot \bar{S}0 \cdot I2 + S1 \cdot S0 \cdot I3$

4. Demultiplexer (DEMUX)

Purpose: Routes single input to one of multiple outputs

Example - 1:4 DEMUX:

- **Input**: D
- **Select lines**: S1, S0
- **Outputs**: Y0, Y1, Y2, Y3

Logic:

- $Y0 = D \cdot \bar{S}1 \cdot \bar{S}0$
- $Y1 = D \cdot \bar{S}1 \cdot S0$
- $Y2 = D \cdot S1 \cdot \bar{S}0$
- $Y3 = D \cdot S1 \cdot S0$

5. Encoder

Purpose: Converts multiple inputs to binary code

Example - 4:2 Encoder:

- **Inputs**: D0, D1, D2, D3 (only one active at a time)
- **Outputs**: A1, A0

Logic:

- $A0 = D1 + D3$
- $A1 = D2 + D3$

6. Decoder

Purpose: Converts binary code to activate one of multiple outputs

Example - 2:4 Decoder:

- **Inputs**: A1, A0
- **Outputs**: Y0, Y1, Y2, Y3

Logic:

- $Y0 = \bar{A}1 \cdot \bar{A}0$
- $Y1 = \bar{A}1 \cdot A0$
- $Y2 = A1 \cdot \bar{A}0$

- Y3 = A1·A0

7. Magnitude Comparator

Purpose: Compares two binary numbers

Example - 2-bit Comparator:

- **Inputs**: A1A0, B1B0
- **Outputs**: A>B, A=B, A<B

Logic:

(uses XNOR gate)

- A>B = A1·$\bar{B}$1 + (A1⊙B1)·A0·$\bar{B}$0
- A=B = (A1⊙B1)·(A0⊙B0)
- A<B = $\bar{A}$1·B1 + (A1⊙B1)·$\bar{A}$0·B0

Design Process

Step 1: Problem Definition

Clearly define what the circuit should accomplish, identify inputs and outputs.

Step 2: Truth Table Creation

List all possible input combinations and determine the desired output for each.

Step 3: Boolean Expression

Derive Boolean expressions from the truth table using: - **Sum of Products (SOP)**: OR of AND terms - **Product of Sums (POS)**: AND of OR terms

Step 4: Simplification

Use Boolean algebra, Karnaugh maps, or other methods to minimize the expression.

Step 5: Implementation

Convert the simplified expression into logic gates.

Example Design: 3-Input Majority Circuit

Problem: Output should be 1 when majority of inputs are 1.

Truth Table:

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Boolean Expression: $Y = A \cdot B \cdot \bar{C} + A \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot C + A \cdot B \cdot C$

Simplified: $Y = A \cdot B + A \cdot C + B \cdot C$

Real-World Applications

1. Arithmetic Logic Unit (ALU)

- Central component in processors
- Performs arithmetic and logical operations
- Uses adders, comparators, and logic gates

2. Address Decoding

- In memory systems to select specific memory locations
- Uses decoders to activate memory chips

3. Data Routing

- Multiplexers in communication systems
- Route data based on control signals

4. Display Systems

- 7-segment display decoders
- Convert binary numbers to display patterns

5. Control Systems

- Priority encoders for interrupt handling
- Comparators for threshold detection

6. Error Detection

- Parity generators and checkers
- XOR gates for simple error detection

This tutorial was generated with Claude AI(accessed: 2025-09-01). Modified and reviewed by JACH for CMSC 132.